Non-Premium Spotify Accounts: Behavior, Dev vs Production

Oct 14, 2026 · @Infrasity Team

What a free account sees

Sign-in and browsing work: playlists, Liked Songs, memories. The app only reads `id` from `/me`, so nothing breaks there.

When the player starts, Spotify's SDK fires `account_error`. The code sets `isPremium = false`, and the play button becomes a green "OPEN IN SPOTIFY" button that opens the track in Spotify. That fallback already works.

The rough edges:

1. **Clicking play too early.** If play is clicked before `account_error` fires, the app waits up to 10 seconds for a device that never appears, then shows "Couldn't connect to Spotify. Try again." A direct 403 shows "Forbidden (Missing Scopes)", which is wrong. Fix: check `product` from `/me` up front.
2. **The queue looks real but isn't.** A 403 on queue is only logged, and the track is still added to the local queue list.
3. **Other controls fail silently.** Only the play button is swapped. Pause, volume and seek fail with no message, and `store.error` is never shown.
4. **Focus reconnects.** Each time the tab regains focus, `player.connect()` runs again and triggers another `account_error`. This is SDK traffic, not the proxy, so the limiter doesn't count it. Harmless but wasteful.

None of these can cause a lockout loop: 403s are never retried at any level. The free path is ugly, not dangerous.

Dev vs production

Mode	Free-account behavior	Notes
Mock mode (default `npm run dev`)	Never shown	Mock user is hardcoded to `premium`; the stub player always emits `ready`. Everything tested here is the Premium experience.
Live dev (`SPOTIFY_LIVE=1`)	Same as production	The only way today to see the free path: sign in with a free account on the app's user list.
Production	Depends on her account	If she is Premium, none of the rough edges matter to her. If not, she gets the "Open in Spotify" fallback everywhere.

Correction to the deploy notes

The session summary says the Spotify client id **and secret** go into Vercel. The app uses PKCE, so there is no secret; only `NEXT_PUBLIC_SPOTIFY_CLIENT_ID` is needed.

Because it is a `NEXT_PUBLIC_*` variable, it is baked in at build time. Set App A's id in Vercel before the first build, and redeploy if it ever changes.

Open questions

- [x] ~~Is her Spotify account Premium? This decides whether the free-account fixes come now or after the request audit and budgets.~~
- [x] ~~Will App A and App B live under the same Spotify developer account? If so, they may share one quota.~~
- [x] ~~Who owns App A? That person must keep Premium active, or the app stops working for everyone.~~

Answers (Oct 14): both accounts are Premium, you own App A, and all apps sit under your one developer account. That makes the shared Spotify quota the main risk: dev and prod budgets must add up to less than Spotify's limit.

Reminder: free-account work at production release

Free users must get a smooth experience too. This is deferred until the app is live, then tested with your second (free) Spotify account.

- [ ] Add the second account to App A's user list in the Spotify dashboard
- [ ] Run Prompt F (free-account mode), after Prompts P and U and the music-outside-Music-World decision
- [ ] Decide what play does for free users: Open in Spotify, or Spotify's embed player (test it live first)
- [ ] Sign in on the real domain with the free account and check: no broken or silent controls, the one-time note shows, play behaves as decided
- [ ] Do the live test while she is unlikely to be using the site (shared quota)